

# sql isolation anomalies

## Аномалии изоляции в sql

База: MySQL 8.0

Аномалии: dirty read, non-repeatable read, phantom read, lost update

### Запуск

```
docker compose up -d
```

```
docker compose exec mysql mysql -uroot -proot isolation_lab
```

Для работы необходимо открыть два терминала и подключиться к базе в каждом:

```
docker compose exec mysql mysql -uroot -proot isolation_lab
```

Перед каждым примером сбрасывать данные:

```
docker compose exec -T mysql mysql -uroot -proot isolation_lab <  
sql/setup.sql
```

### Dirty read

Одна транзакция читает данные, которые другая транзакция еще не зафиксировала.

Терминал 1:

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;  
  
START TRANSACTION;
```

```
UPDATE accounts SET balance = 200 WHERE id = 1;
```

```
SELECT * FROM accounts WHERE id = 1;
```

Терминал 2:

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

```
START TRANSACTION;
```

```
SELECT * FROM accounts WHERE id = 1;
```

```
COMMIT;
```

Терминал 1:

```
ROLLBACK;
```

```
SELECT * FROM accounts WHERE id = 1;
```

Результат: второй терминал увидел баланс 200, хотя после роллбэк в базе снова 500.

```

mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql>
mysql> UPDATE accounts SET balance = 200 WHERE id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SELECT * FROM accounts WHERE id = 1;
+----+-----+-----+
| id | owner_name | balance |
+----+-----+-----+
|  1 | Ivan      |      200 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM accounts WHERE id = 1;
+----+-----+-----+
| id | owner_name | balance |
+----+-----+-----+
|  1 | Ivan      |      500 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql>

```

Терминал 1

```
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql>
mysql> SELECT * FROM accounts WHERE id = 1;
+----+-----+-----+
| id | owner_name | balance |
+----+-----+-----+
|  1 | Ivan      |      200 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

Терминал 2

Скриншот: добавить сюда.

Как избежать: не использовать READ UNCOMMITTED; достаточно READ COMMITTED или более строгого уровня.

## Non-repeatable read

Идея: внутри одной транзакции повторное чтение той же строки возвращает другое значение.

Терминал 1:

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;
```

```
SELECT balance FROM accounts WHERE id = 1;
```

Терминал 2:

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
START TRANSACTION;
```

```
UPDATE accounts SET balance = 700 WHERE id = 1;
```

```
COMMIT;
```

Терминал 1:

```
SELECT balance FROM accounts WHERE id = 1;
```

```
COMMIT;
```

Результат: первый терминал сначала получил 500, потом 700

```
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql>  
mysql> START TRANSACTION;  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql>  
mysql>  
mysql>  
mysql> SELECT balance FROM accounts WHERE id = 1;  
+-----+  
| balance |  
+-----+  
|      500 |  
+-----+  
1 row in set (0.00 sec)
```

```
mysql> SELECT balance FROM accounts WHERE id = 1;  
+-----+  
| balance |  
+-----+  
|      700 |  
+-----+  
1 row in set (0.00 sec)
```

```
mysql>  
mysql> COMMIT;  
Query OK, 0 rows affected (0.00 sec)  
  
mysql>
```

Терминал 1

```
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql>
mysql> UPDATE accounts SET balance = 700 WHERE id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> █
```

Терминал 2

Как избежать: использовать REPEATABLE READ или читать строку с блокировкой через SELECT ... FOR UPDATE

## Phantom read

Идея: внутри одной транзакции повторный запрос по условию возвращает другое количество строк.

Терминал 1:

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;

SELECT COUNT(*) AS new_orders FROM orders WHERE status = 'new';
```

Терминал 2:

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;  
  
START TRANSACTION;  
  
INSERT INTO orders (account_id, amount, status)  
VALUES (2, 150, 'new');  
  
COMMIT;
```

Терминал 1:

```
SELECT COUNT(*) AS new_orders FROM orders WHERE status = 'new';  
  
COMMIT;
```

Результат: первый терминал сначала получил 2, потом 3.





```
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql>
mysql> INSERT INTO orders (account_id, amount, status)
      ->
      -> VALUES (2, 150, 'new');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql>
```

Терминал 2

Как избежать: использовать REPEATABLE READ / SERIALIZABLE или блокировки диапазона, если запрос должен видеть стабильный набор строк.

## Lost update

Идея: две транзакции читают одно значение, считают новое значение отдельно и последняя запись затирает первую.

Терминал 1:

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;


SELECT balance FROM accounts WHERE id = 1;
```

Терминал 2:

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;  
  
START TRANSACTION;  
  
SELECT balance FROM accounts WHERE id = 1;
```

Терминал 1:

```
UPDATE accounts SET balance = 200 WHERE id = 1;  
  
COMMIT;
```

Терминал 2:

```
UPDATE accounts SET balance = 800 WHERE id = 1;  
  
COMMIT;  
  
SELECT * FROM accounts WHERE id = 1;
```

Результат: обе транзакции читали 500. Первая списала 300, вторая прибавила 300, но итог стал 800. Ожидаемый итог после двух операций: 500.

```
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql>
mysql> SELECT balance FROM accounts WHERE id = 1;
+-----+
| balance |
+-----+
|      700 |
+-----+
1 row in set (0.00 sec)

mysql> UPDATE accounts SET balance = 200 WHERE id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

Терминал 1

```

mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql>
mysql> SELECT balance FROM accounts WHERE id = 1;
+-----+
| balance |
+-----+
|      700 |
+-----+
1 row in set (0.00 sec)

mysql> UPDATE accounts SET balance = 800 WHERE id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql>
mysql>
mysql> SELECT * FROM accounts WHERE id = 1;
+----+-----+-----+
| id | owner_name | balance |
+----+-----+-----+
|  1 | Ivan      |      800 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> 

```

Терминал 2

Как избежать: обновлять значение атомарно ( $balance = balance + 300$ ), использовать SELECT ... FOR UPDATE или версиюность строки.